Practical no : 5

Database Structure :

- **Student** (StudentID, Name, DOB, DeptID, Semester, Email)
- **Department** (DeptID, DeptName, HOD, Building)
- **Faculty** (FacultyID, FName, DeptID, Designation, Salary)
- **Course** (CourseID, CourseName, DeptID, Credits, FacultyID)
- **Enrollment** (EnrollID, StudentID, CourseID, Semester, Marks, Grade)

Database Setup and Data Insertion :

```
import sqlite3
import pandas as pd

# Connect to SQLite database (creates university.db file)
conn = sqlite3.connect('university.db')
cursor = conn.cursor()

# 1. Create Tables
cursor.execute("DROP TABLE IF EXISTS Enrollment;")
cursor.execute("DROP TABLE IF EXISTS Course;")
cursor.execute("DROP TABLE IF EXISTS Faculty;")
cursor.execute("DROP TABLE IF EXISTS Student;")
cursor.execute("DROP TABLE IF EXISTS Department;")

cursor.execute('''
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
```

```
    DeptName VARCHAR(50),
    HOD VARCHAR(50),
    Building VARCHAR(50)
);
''')

cursor.execute('''
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    DOB DATE,
    DeptID INT,
    Semester INT,
    Email VARCHAR(50),
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
''')

cursor.execute('''
CREATE TABLE Faculty (
    FacultyID INT PRIMARY KEY,
    FName VARCHAR(50),
    DeptID INT,
    Designation VARCHAR(50),
    Salary INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
''')
```

```
cursor.execute('''
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50),
    DeptID INT,
    Credits INT,
    FacultyID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID),
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
);
''')

cursor.execute('''
CREATE TABLE Enrollment (
    EnrollID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    Semester INT,
    Marks INT,
    Grade VARCHAR(5),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
''')
```

```
# 2. Insert Data

departments = [(101, 'Computer Applications', 'Dr. Sharma', 'Block A'), (102, 'Computer Science', 'Dr. Verma', 'Block B'), (103, 'Electronics', 'Dr. Iyer', 'Block C'), (104, 'Mathematics', 'Dr. Rao', 'Block D')]

students = [(1, 'Amit Kumar', '2002-05-14', 101, 3, 'amit@univ.edu'), (2, 'Priya Singh', '2002-08-21', 101, 3, 'priya@univ.edu'), (3, 'Rahul Verma', '2001-11-02', 102, 3, 'rahul@univ.edu'), (4, 'Sneha Joshi', '2002-01-17', 101, 5, 'sneha@univ.edu'), (5, 'Karan Mehta', '2001-07-09', 103, 3, 'karan@univ.edu'), (6, 'Neha Gupta', '2002-03-25', 102, 5, 'neha@univ.edu'), (7, 'Arjun Nair', '2001-12-30', 101, 3, 'arjun@univ.edu'), (8, 'Divya Reddy', '2002-09-11', 104, 3, 'divya@univ.edu')]

faculties = [(201, 'Dr. Anil Deshpande', 101, 'Professor', 95000), (202, 'Ms. Kavita Rane', 101, 'Assistant Professor', 55000), (203, 'Dr. Suresh Pillai', 102, 'Associate Professor', 75000), (204, 'Mr. Rohit Kulkarni', 103, 'Assistant Professor', 50000), (205, 'Dr. Meena Iyengar', 101, 'Professor', 98000)]

courses = [(301, 'DBMS', 101, 4, 201), (302, 'Java Programming', 101, 4, 202), (303, 'Computer Networks', 102, 3, 203), (304, 'Digital Electronics', 103, 3, 204), (305, 'Software Engineering',101, 3, 205), (306, 'Operating Systems', 102, 4, 203)]

enrollments = [(1001, 1, 301, 3, 82, 'A'), (1002, 1, 302, 3, 78, 'B+'), (1003, 2, 301, 3, 91, 'A+'), (1004, 2, 305, 3, 68, 'B'), (1005, 3, 303, 3, 74, 'B+'), (1006, 4, 301, 5, 88, 'A'), (1007, 5, 304, 3, 55, 'C'), (1008, 6, 306, 5, 79, 'B+'), (1009, 7, 302, 3, 93, 'A+'), (1010, 8, 301, 3, 60, 'C+')]


cursor.executemany("INSERT INTO Department VALUES (?, ?, ?, ?);", departments)

cursor.executemany("INSERT INTO Student VALUES (?, ?, ?, ?, ?, ?);", students)

cursor.executemany("INSERT INTO Faculty VALUES (?, ?, ?, ?, ?);", faculties)

cursor.executemany("INSERT INTO Course VALUES (?, ?, ?, ?, ?);", courses)

cursor.executemany("INSERT INTO Enrollment VALUES (?, ?, ?, ?, ?, ?);", enrollments)


conn.commit()

print("Database initialized, tables created, and data inserted successfully!")
```

```
... Database initialized, tables created, and data inserted successfully!
```

Question based on Views

1. Create a view named StudentInfo that displays StudentID, Name, and DeptName by joining the Student and Department tables.

```
cursor.execute("DROP VIEW IF EXISTS StudentInfo;")
cursor.execute('''
CREATE VIEW StudentInfo AS
SELECT s.StudentID, s.Name, d.DeptName
FROM Student s
JOIN Department d ON s.DeptID = d.DeptID;
''')
conn.commit()

# Display results
print("--- StudentInfo View ---")
print(pd.read_sql_query("SELECT * FROM StudentInfo;", conn))
```

```
--- StudentInfo View ---
   StudentID        Name             DeptName
0          1  Amit Kumar  Computer Applications
1          2  Priya Singh  Computer Applications
2          3  Rahul Verma      Computer Science
3          4  Sneha Joshi  Computer Applications
4          5  Karan Mehta          Electronics
5          6   Neha Gupta      Computer Science
6          7   Arjun Nair  Computer Applications
7          8  Divya Reddy          Mathematics
```

2. Create a view TopScorers that shows StudentID, Name, CourseName, and Marks of students who scored more than 75 marks in any course.

```
cursor.execute("DROP VIEW IF EXISTS TopScorers;")
cursor.execute('''
CREATE VIEW TopScorers AS
SELECT e.StudentID, s.Name, c.CourseName, e.Marks
FROM Enrollment e
JOIN Student s ON e.StudentID = s.StudentID
JOIN Course c ON e.CourseID = c.CourseID
WHERE e.Marks > 75;
''')
conn.commit()

print("\n--- TopScorers View ---")
print(pd.read_sql_query("SELECT * FROM TopScorers;", conn))
```

```
...
--- TopScorers View ---
   StudentID        Name        CourseName  Marks
0          1   Amit Kumar              DBMS     82
1          1   Amit Kumar   Java Programming     78
2          2  Priya Singh              DBMS     91
3          4  Sneha Joshi              DBMS     88
4          6   Neha Gupta  Operating Systems     79
5          7   Arjun Nair   Java Programming     93
```

3. Create a view FacultyDeptView displaying FacultyID, FName, Designation, and DeptName for all faculty members.

```
cursor.execute("DROP VIEW IF EXISTS FacultyDeptView;")
cursor.execute('''
CREATE VIEW FacultyDeptView AS
```

```
SELECT f.FacultyID, f.FName, f.Designation, d.DeptName
FROM Faculty f
JOIN Department d ON f.DeptID = d.DeptID;
''')
conn.commit()

print("\n--- FacultyDeptView ---")
print(pd.read_sql_query("SELECT * FROM FacultyDeptView;", conn))
```

```
...
    --- FacultyDeptView ---
       FacultyID              FName          Designation              DeptName
    0        201  Dr. Anil Deshpande            Professor  Computer Applications
    1        202     Ms. Kavita Rane  Assistant Professor  Computer Applications
    2        203   Dr. Suresh Pillai  Associate Professor       Computer Science
    3        204  Mr. Rohit Kulkarni  Assistant Professor            Electronics
    4        205   Dr. Meena Iyengar            Professor  Computer Applications
```

4. Write a query to insert a new record through the StudentInfo view and observe what happens if it involves multiple base tables.

```
print("\n--- Q4 Observation ---")
try:
    cursor.execute("INSERT INTO StudentInfo VALUES (9, 'Rohan Das', 'Computer Science');")
except Exception as e:
    print(f"Error encountered: {e}")
    print("Observation: In relational databases (including SQLite), you cannot directly insert records into a view built on multiple base tables (a complex view involving a JOIN) because the engine cannot determine how to split the incoming columns safely across the underlying tables.")
```

```
--- Q4 Observation ---
Error encountered: cannot modify StudentInfo because it is a view
Observation: In relational databases (including SQLite), you cannot directly insert records into a view built on multiple base tables (a complex view involving a JOIN) becaus
```

5. Create a view CourseEnrollmentCount that shows CourseID, CourseName, and the total number of students enrolled in each course (use GROUP BY).

```
cursor.execute("DROP VIEW IF EXISTS CourseEnrollmentCount;")
cursor.execute('''
CREATE VIEW CourseEnrollmentCount AS
SELECT c.CourseID, c.CourseName, COUNT(e.StudentID) AS TotalStudents
FROM Course c
LEFT JOIN Enrollment e ON c.CourseID = e.CourseID
GROUP BY c.CourseID, c.CourseName;
''')
conn.commit()

print("\n--- CourseEnrollmentCount View ---")
print(pd.read_sql_query("SELECT * FROM CourseEnrollmentCount;", conn))
```

```
--- CourseEnrollmentCount View ---
   CourseID           CourseName  TotalStudents
0       301                 DBMS              4
1       302      Java Programming             2
2       303     Computer Networks             1
3       304   Digital Electronics             1
4       305  Software Engineering             1
5       306     Operating Systems             1
```

6. Create a view DepartmentStrength showing DeptName and the total number of students in each department.

```
cursor.execute("DROP VIEW IF EXISTS DepartmentStrength;")
cursor.execute('''
CREATE VIEW DepartmentStrength AS
SELECT d.DeptName, COUNT(s.StudentID) AS TotalStudents
FROM Department d
```

```
LEFT JOIN Student s ON d.DeptID = s.DeptID
GROUP BY d.DeptID, d.DeptName;
''')
conn.commit()

print("\n--- DepartmentStrength View ---")
print(pd.read_sql_query("SELECT * FROM DepartmentStrength;", conn))
```

```
"""
--- DepartmentStrength View ---
              DeptName  TotalStudents
0  Computer Applications             4
1       Computer Science             2
2            Electronics             1
3            Mathematics             1
```

7. Modify (using CREATE OR REPLACE VIEW / ALTER VIEW) the StudentInfo view to also include the Semester column.

```
cursor.execute("DROP VIEW IF EXISTS StudentInfo;")
cursor.execute('''
CREATE VIEW StudentInfo AS
SELECT s.StudentID, s.Name, s.Semester, d.DeptName
FROM Student s
JOIN Department d ON s.DeptID = d.DeptID;
''')
conn.commit()

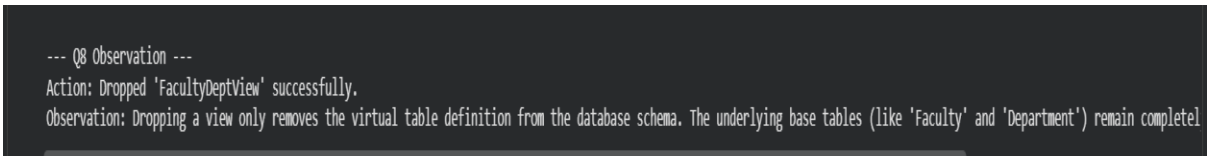
print("\n--- Modified StudentInfo View ---")
print(pd.read_sql_query("SELECT * FROM StudentInfo;", conn))
```

```
"""
--- Modified StudentInfo View ---
   StudentID         Name  Semester              DeptName
0          1   Amit Kumar         3  Computer Applications
1          2  Priya Singh         3  Computer Applications
2          3  Rahul Verma         3       Computer Science
3          4  Sneha Joshi         5  Computer Applications
4          5  Karan Mehta         3            Electronics
5          6   Neha Gupta         5       Computer Science
6          7   Arjun Nair         3  Computer Applications
7          8  Divya Reddy         3            Mathematics
```

8. Write a query to drop a view and explain what happens to the underlying base tables.

```
cursor.execute("DROP VIEW IF EXISTS FacultyDeptView;")
conn.commit()


print("\n--- Q8 Observation ---")
# Verify base tables still exist
try:
    check_table = pd.read_sql_query("SELECT * FROM Faculty LIMIT 1;", conn)
    print("Action: Dropped 'FacultyDeptView' successfully.")
    print("Observation: Dropping a view only removes the virtual table definition from the database schema. The underlying base tables (like 'Faculty' and 'Department') remain completely unaffected, and no actual data is deleted.")
except Exception as e:
    print(f"Error checking base table: {e}")
```



9. Create a view that displays only Semester III students along with their enrolled courses (a multi-table join view).

```
cursor.execute("DROP VIEW IF EXISTS SemesterThreeCourses;")
cursor.execute('''
CREATE VIEW SemesterThreeCourses AS
SELECT s.StudentID, s.Name, s.Semester, c.CourseName
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course c ON e.CourseID = c.CourseID
WHERE s.Semester = 3;
```

```
''')
conn.commit()

print("\n--- SemesterThreeCourses View ---")
print(pd.read_sql_query("SELECT * FROM SemesterThreeCourses;", conn))
```

```
--- SemesterThreeCourses View ---
   StudentID        Name  Semester          CourseName
0          1   Amit Kumar         3                DBMS
1          1   Amit Kumar         3     Java Programming
2          2  Priya Singh         3                DBMS
3          2  Priya Singh         3  Software Engineering
4          3  Rahul Verma         3    Computer Networks
5          5  Karan Mehta         3   Digital Electronics
6          7   Arjun Nair         3     Java Programming
7          8  Divya Reddy         3                DBMS
```

10. Differentiate between a **simple view** and a **complex view** by creating one example of each from the above schema.

```
print("\n--- Q10 Explanation & Example ---")
print("1. Simple View: Created from a single table. It does not contain joins, group functions, or distinct criteria. It usually allows DML actions (INSERT, UPDATE, DELETE).")
print("   Example: A view showing only high-salary faculty.")
cursor.execute("DROP VIEW IF EXISTS HighSalaryFaculty;")
cursor.execute("CREATE VIEW HighSalaryFaculty AS SELECT FacultyID, FName, Salary FROM Faculty WHERE Salary > 70000;")
print(pd.read_sql_query("SELECT * FROM HighSalaryFaculty;", conn))

print("\n2. Complex View: Created using multiple tables (JOINs), contains aggregate functions (SUM, COUNT, AVG), or GROUP BY statements. DML operations are generally restricted.")
print("   Example: 'DepartmentStrength' view (uses JOIN and COUNT/GROUP BY) created in Q6.")
```

Clean up connection at the very end

conn.close()

```
"""
    --- Q10 Explanation & Example ---
    1. Simple View: Created from a single table. It does not contain joins, group functions, or distinct criteria. It usually allows DML actions (INSERT, UPDATE, DELETE).
       Example: A view showing only high-salary faculty.
       FacultyID              FName  Salary
    0        201  Dr. Anil Deshpande   95000
    1        203   Dr. Suresh Pillai   75000
    2        205   Dr. Meena Iyengar   98000

    2. Complex View: Created using multiple tables (JOINs), contains aggregate functions (SUM, COUNT, AVG), or GROUP BY statements. DML operations are generally restricted.
       Example: 'DepartmentStrength' view (uses JOIN and COUNT/GROUP BY) created in Q6.
```